

## Q.no 10

| DS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Session | Stack | Question Information | Level 1 | Challenge 50 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-------|----------------------|---------|--------------|
| <p><b>Question description</b></p> <p>Hassan gets a job in a software company in Hyderabad. The training period for the first three months is 20000 salary. Then incremented to 25000 salaries.</p> <p>Training is great but they will give you a programming task every day in three months. Hassan must finish it in the allotted time. His teammate Jocelyn gives him a task to complete the concept of Postfix to Prefix Conversion for a given expression. can you help him?</p> <p><b>Functional Description:</b></p> <ul style="list-style-type: none"> <li>Read the Prefix expression in reverse order (from right to left)</li> <li>If the symbol is an operand, then push it onto the Stack</li> <li>If the symbol is an operator, then pop two operands from the Stack</li> </ul> <p>Create a string by concatenating the two operands and the operator after them.</p> <p><b>string = operand1 + operand2 + operator</b></p> <p>And push the resultant string back to Stack</p> <ul style="list-style-type: none"> <li>Repeat the above steps until end of Prefix expression.</li> </ul> <p><b>Constraints</b></p> <p>the input should be a expressions</p> <p><b>Input Format</b></p> <p>Single line represents the postfix expressions</p> <p><b>Output Format</b></p> <p>Single line represents the prefix expression</p> |         |       |                      |         |              |

### Logical Test Cases

|                                                                                                             |                                                                                                     |
|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <p><b>Test Case 1</b></p> <p>INPUT (STDIN)</p> <p>ABC/-AK/L-*</p> <p>EXPECTED OUTPUT</p> <p>*-A/BC-/AKL</p> | <p><b>Test Case 2</b></p> <p>INPUT (STDIN)</p> <p>AB+CD-*</p> <p>EXPECTED OUTPUT</p> <p>*+AB-CD</p> |
|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|

### Mandatory Test Cases

|                                                                                   |
|-----------------------------------------------------------------------------------|
| <p><b>Test Case 1</b></p> <p>KEYWORD</p> <p>string postToPre(string post_exp)</p> |
|-----------------------------------------------------------------------------------|

### Complexity Test Cases

|                                                                 |                                                         |                                                 |
|-----------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------|
| <p><b>Test Case 1</b></p> <p>CYCLOMATIC COMPLEXITY</p> <p>5</p> | <p><b>Test Case 2</b></p> <p>TOKEN COUNT</p> <p>220</p> | <p><b>Test Case 3</b></p> <p>NLOC</p> <p>46</p> |
|-----------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------|

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char stack[100][100];
int top = -1;
```

```

void push(char str[])
{
    top++;
    strcpy(stack[top], str);
}

void pop(char str[])
{
    strcpy(str, stack[top]);
    top--;
}

void postToPre(char post_exp[])
{
    int i;
    char op1[100], op2[100], temp[100];

    for(i = 0; post_exp[i] != '\0'; i++)
    {
        if(isalnum(post_exp[i]))
        {
            temp[0] = post_exp[i];
            temp[1] = '\0';

            push(temp);
        }
        else
        {
            pop(op2);
            pop(op1);

            temp[0] = post_exp[i];
            temp[1] = '\0';

            char result[100];

            strcpy(result, temp);
            strcat(result, op1);
            strcat(result, op2);

            push(result);
        }
    }

    pop(temp);

    printf("%s\n", temp);
}

int main()

```

```
{
char post_exp[100];

scanf("%s", post_exp);

postToPre(post_exp);

return 0;
}
```

Output:

Test\_case 1:

```
AB+CD-*
*+AB-CD%
```

Test\_case 2:

```
ABC/-AK/L-*
*-A/BC-/AKL%
```